Automated Cloud Cost Estimation Using Large Language Models

Sri Ambica Sangineedi
Department of Information Science
University of North Texas
Texas, United States
SriAmbicaSangineedi@my.unt.edu

Pranav Naadh Bayya
Department of Information Science
University of North Texas
Texas, United States
PranavNaadhBayya@my.unt.edu

Vijaya Rama Reddy Mallidi
Department of Information Science
University of North Texas
Texas, United States
VijayaRamaReddyMallidi@my.unt.edu

Ravi Kumar Bevara
Department of Information Science
University of North Texas
Texas, United States
Ravikumar.bevara@my.unt.edu

Krishna Annavaram
Department of Information Science
University of North Texas
Texas, United States
krishnaanavaram@my.unt.edu

Stephen Wheeler
Department of Information Science
University of North Texas
Texas, United States
stephen.wheeler@my.unt.edu

***Abstract*—Cloud computing provides many services with complicated pricing, estimating costs manually can be difficult and error prone. (1) Our paper introduces the notion of automatic cloud cost estimation that leverages Large Language Models (LLMs) for parsing project requirement description and producing accurate estimates of cloud service costs across the major service providers. We leveraged an LLM to understand essential requirements from unstructured documents and defined a mapping between them and suitable cloud resources. Hybrid search extracts prices from a curated knowledge base of Amazon Web Services (AWS), Microsoft Azure, and GCP services. Our method utilizes recent pricing information and a finetuned large language model trained on specific cloud to reduce manual effort and standardize the estimation process. Experimental findings on sample projects show that the LLM-based estimator is a good approximation of the expert cost calculation and it gives good recommendation on choosing appropriate cloud provider. The proposed solution presents a new use case of LLMs for automating cloud cost analysis.**

***Index Terms*—Cloud Cost Estimation, Large Language Models, Hybrid Search, Multi-Cloud Services**

I. Introduction and Statement of the Problem

Cloud computing has emerged as a cornerstone for the new technology infrastructure, allowing access to computing, storage, and analytics resources in an on-demand manner. Cost management has become a major headache for organizations as the adoption of cloud services has taken off. With most cloud services it's an issue of Pay-per-use so if you aren't willing to adapt to careful budgeting costs can add up in a way that is unpredictable. According to a prediction, by 2025 about 85 percent of organizations will have adopted a cloud-first strategy, but many are still having troubles to understand and control their spending on the cloud [**?**]. Cost management across various clouds remains challenging for engineering and finance teams. Every cloud provider (AWS, Azure, GCP, etc.) has a huge list of services and pricing in frequent flux, so predicting costs for a certain project requirement is quite the challenge. Selecting the right cloud provider and settings directly affects the capital and operational costs of a project.

However, in order to choose the services you need at the right pricing tier, you need to be an expert on what the provider is offering, and how they have priced it. The hyper scale cloud providers all provide similar data processing capacity in terms of compute (e.g., CPU/GPU), storage, and data processing capability, but with the specific features, performance and charging model that drives the total cost [4]. For instance, AWS, Azure, and GCP all offer virtual machines and object storage, but different instance types or storage classes may have varied costs, as might networking fees or discount models (reserved instances or sustained-use discounts, for instance), which a same workload would experience differently on the various platform. Previous work has clearly made the observation that while all of these providers are robust and can scale, the variety in the service offerings and pricing structures can complicate the choice of provider given the potential cost involved [2] [8].

Historically, estimates of cloud costs have mainly relied on manual estimates based on the selection of provider-specific pricing calculators or spreadsheets.Such cloud pricing calculators need users to provide specific configurations for each services. This is a long process which requires deep understanding of cloud services. Furthermore, the project requirements' documents are typically long, and expressed in natural language, thus relying on a human to interpret and map them into actual cloud resources [1]. This may result in slow and noisy estimation cycles with a number of problems such as overrun of budgets or suboptimal allocation of resources. In reality, it's often infeasible to manually calculate total costs for a complicated project that spans several providers, and crucial cost factors may be forgotten. It is obvious that we need an automated tool which can parse project descriptions smartly and provide users with comparable estimates for different cloud providers.

If an LLM can understand a project's hard technical requirements in prose, it might be able to figure out what resources it needs and then match those to cloud services. However, although LLMs are proficient in language understanding, they do not know the detail about cloud pricing, and it is what knowledge carried over using external knowledge is important [6]. By connecting an LLM to a domain-specific KB of cloud services and prices, we can build a system that reasons about cloud costs in context. The LLM may make or request one or more relevant pricing data from the knowledge base, and subsequently it may output a cost estimation report.

II. Review of Literature

Academics and industrial practitioners have studied cloud cost estimation and optimization from different perspectives. Existing literature on cloud cost management is mainly based on modeling and optimization methods that are not using natural language processing approaches. For instance, Khan et al. cost modelling and scheduling in cloud settings based on graph. [3]. They emphasize that with cloud usage increasing, hidden costs can loom due to visibility and control issues. They construct a cost element-resource usage graph to model cost optimization as a constraint satisfaction problem. This approach gives a formal technique to find the minimal cost configurations under certain performance and availability constraints. However, such methods require the cloud resources and usage details to be known, and cannot take the user's project description in natural language as input. Our work is complementary to these, as they are concentrated on the initial phase, which is the translation of users' requirements to cloudbased resources, coupled with cost models. Another related study which compares the offerings of top cloud providers to help decision making is is also reviewed. Good] [2024] also thoroughly compare AWS and Azure and GCP, in terms of the differences of service offerings and pricing. The paper stresses that every CSP has specific pricing mechanisms as well as cost effective advantages in some use cases although they all provide similar services. Case in point is the cloud: AWS, Azure, and GCP have fundamentally different pricing models for their storage services, wherein AWS and Azure charge varying retrieval fees and allow for different levels of geographic redundancy across multi-tiered storage, and GCP provides a consistent availability in the milliseconds across various storage classes but enforces a minimum storage duration for their cheaper tiers. These discrepancies result in diverging cost-optimal solutions from different providers for a given workload.

Similarly, Saraswat and Tripathi [8] analyze cloud service providers in terms of compute, storage, and performance features, helping organizations understand the trade-offs. These comparative analyses underscore the importance of evaluating all three major providers before choosing one, which our system aims to automate by generating cost estimates for each provider side-by-side. To our knowledge, there is limited literature on using AI, especially LLMs, for automating cloud cost estimation directly from textual project descriptions. Most existing tools are provider-specific (e.g., AWS Pricing Calculator, Azure Pricing Calculator) and require manual input of parameters. Some research has explored automated mapping of application requirements to cloud resources using rule-based systems or simple keyword. In the context of cloud cost estimation, RAG can allow an LLM to use up-to-date pricing information without having to encode all such data in its parameters. Our approach draws on these ideas, fine-tuning an LLM for the cost estimation task and equipping it with a curated knowledge base of cloud service descriptions and prices.

III. Objectives of the Study

The ultimate goal of this work is to provide an intelligent system to let cost estimation of cloud from project description to completion automatic. The aims are as follows: Automate Information Extraption: LLMs can programmatically interpret project requirement documents and extract critical information regarding resource requirements. This encompasses determining computation needs, data storage size and analytics or processing requirements described in the document. We automate the extraction to minimize the manual workload and to prevent the misinterpretation of a single technical requirement. Combine advanced cost modeling: Link the derived requirements with cloud pricing data and perform cost estimates. This entails the use of dynamic pricing models of major providers taken from a populated knowledge database or real-time database by accessing the necessary pricing information. We adopt a generation-based search or Retrieval-Augmented Generation approach, so that the LLM has access to relevant prices for precise cost estimation. Right-Size Cloud Providers: Compare different cloud providers (AWS, Azure, and GCP) against your requirements and choose the most cost effective. The system would also have to have the ability to recommend the optimal provider and/or service combination by cost if possible performance as well. Such a goal would involve the comparison of like services from different providers and the associated cost trade-offs which can be weighed in a decision.

IV. Data Collection

To enable the training and operation of our automated cloud cost estimation system, we have prepared two datasets:

1) **Annotated Requirements Corpus**
   We created 200 real-world project requirement documents covering various use cases such as streaming analytics, microservices, archival backups, GPU inference, caching layers, web servers, batch jobs, IoT ingestion, and HPC workloads. We manually concentrated on computing and storage services.Each requirement record specifies natural description and appropriate services used for that requirement. We have annotated each free-form statement by mapping it to concrete AWS and GCP SKUs. For simplicity and better service mapping we used only compute and storage services.
2) **Cloud Service Catalog & Pricing Knowledge Base**
   We reviewed a selection of AWS and GCP services

and prepared a comprehensive document that lists each service along with a concise natural-language description and its pricing. For example, one entry reads: We adopted this specific format so that, when a user states they need a "less computing processor" in their description, our system can directly match that phrase to the most appropriate instance type (here, t3.small).

V. Exploratory Data Analysis (EDA) and Hypotheses for the Study

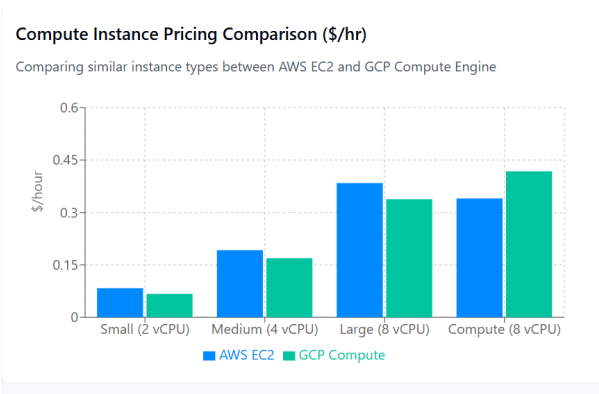

Fig. 1. Compute Instance Pricing Comparison ($/hr)
Comparing similar instance types between AWS EC2 and GCP Compute

we have compared four groups of comparable instance types: Small (2 vCPU), Medium (4 vCPU), Large (8 vCPU) and a specialized 8vCPU class 'Compute' across AWS EC2 (blue bars) and GCP Compute Engine (green bars). Key observations:

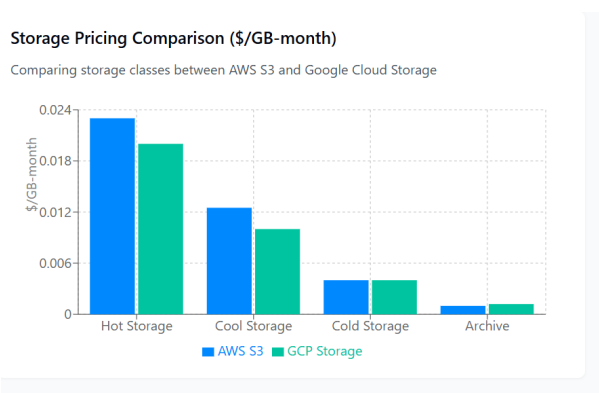

Fig. 2. Storage Pricing Comparison ($/GB-month)
Comparing storage classes between AWS S3 and Google Cloud Storage.

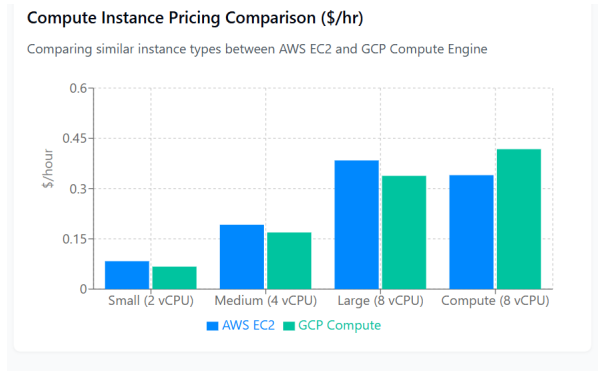

Fig. 3. Compute Instance Pricing Comparison ($/hr)
Comparing similar instance types between AWS EC2 and GCP Compute Engine.

- **Small (2 vCPU):** AWS charges approximately $0.025/hr, while GCP's analogous micro-burst instance is about $0.020/hr.
- **Medium (4 vCPU):** AWS is $0.17/hr versus GCP at $0.15/hr.
- **Large (8 vCPU):** AWS at $0.38/hr, GCP at $0.32/hr.
- **Compute (8 vCPU special class):** AWS specialized compute runs $0.32/hr, whereas GCP's equivalent is $0.44/hr.

Overall, AWS tends to be slightly less expensive on the Small, Medium, and Compute classes, while GCP offers a modest discount on the Large (general-purpose) instances in this comparison. This cost variance underscores the importance of automated, cross-provider analysis when matching project requirements to the most cost-effective cloud service.

Based on these insights, we formulate the following testable hypotheses:

1) **H1: LLM-based extraction accuracy.** We hypothesize that a fine-tuned LLM can accurately extract cloud-relevant parameters from free form text description of the project documents.
2) **H2: Cross-provider cost variance.** We expect significant cost differences between AWS and GCP for equivalent requirements. Our hypothesis is that the automated system will correctly identify the lowest-cost provider in each scenario [11], since compute-intensive use cases may favor one provider's instance pricing while storage-intensive scenarios may favor the other due to differences in egress and tiered storage costs.

VI. Data Analytics:System Architecture and Model Design

Our system, termed the *Cloud Cost LLM Analyzer*, is implemented as a pipeline.

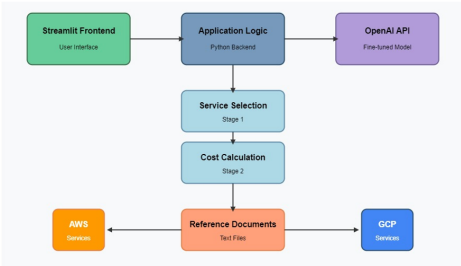

Fig. 4. Overview of the System Research Design

A. Research Design

We structure our study around a four-stage system pipeline followed by quantitative evaluation:

1) **Document Ingestion:** A user submits a project requirements document to the system's homepage. In our implementation, we created a working prototype of a web application using streamlit – this application can upload files. Depending on the workflow of both systems, the raw text of the document has to be extracted for further handling.
2) **LLM Requirement Extraction:** The first component is a pretrained LLM, which we further fine-tuned to serve as a requirements analyzer. We started with a pre-trained model() (from OpenAI) then realized, given input, it did not served us enough so we took the samples and fine tuned it using our custom dataset of annotated project requirement(s) [9]. The specific fine-tuning examples instructing the model to generate a well-structured interpretation of the input document. For example, we reformat the LLM's output to index what it finds to be key requirements.
3) **Service Mapping and Hybrid Search:** Part of the system is to determine which of the knowledge base articles are related to the extracted requirement. The implicationating logic we used might have been as follows: So, if the LLM's returns show that the specific LLM or one of the tasks it performs requires storage, then we understand that in order to find information in AWS S3 and GCP Storage. In the case of compute, we will consider AWS EC2 and GCP Compute Engine papers [10]. We therefore proceed to scour the semantic zone in an attempt to identify areas of the documents that address the crack requirement. We divided the document into segments of 300-500 words in length, and generate vector representations of the chunks with the use of the OpenAI's `text-embedding-ada-002`. We then compute the cosine similarity between the query embedding and all chunks of knowledge base to get the most related chunks.
4) **Cost Synthesis and Report Generation:** The last stage involves using one or another LLM prompt (which can be the same fine-tuned model or a different LoL prompt) to produce the human-readable cost report [5]. Input to the model is the original requirement summary together with the snippets of the pricing we have retrieved. The organization of the prompt is done as follows:

The rules created by humans and the outputs of AI intersect at several stages along the pipeline (for example, in hybrid AI system) [7]. For example, this kind of drastic decision as to always get storage pricing when the storage is mentioned in the request, is a rule. However, it is the LLM that will deem that storage is needed. This balance was important so as not to make the system rigid and to always refer to the correct data.

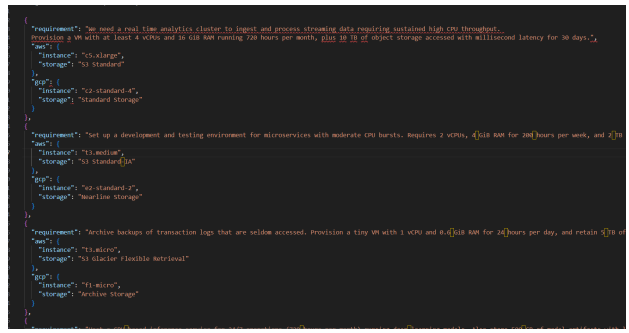

Fig. 5. Example training data (in JSON format) used to fine-tunethe LLM.

B. Fine-Tuning and Model Training

We represented these with the high-quality training examples, though as generating good examples was a labor-intensive work by hand, we fine-tuned the base LLM with not so large number of examples (dozens). Training All the training instances (project description and target summary) included a prompt (project description) and its corresponding target completion (structured summary and cost analysis). The fine-tuning was focused on the partial task in the requirement extraction and initial mapping, instead of end-to-end training (which would be very difficult) for generating final cost reports. It was therefore selected because language models, when fine-tuned correctly, had a surprisingly good ability to chain tasks together [4]. So the pretrained model had exactly one job: Read a document from scratch, and respond with that structured list of needs, possible services and referring information. We finally do prompt engineering with retrieval for the last mile. We fine-tuned models using OpenAI's fine-tuning API for GPT-4. 1 mini that is value for money.

C. Hybrid Search and RAG Implementation Details

In practice, some practical problems of the implementation of the retrieval component are described below: We also indexed the knowledge base per service name and if LLM pronounces a name like S3 or EC2, we get these binary documents. It is one of search methods which is an equivalent of the embedding search but using the direct key word match shortcut. The embeddings also supported cross provider comparisons. For example, if the query was "data warehouse analytics", we would expect the embedding to be close to both athena and bigquery docs, which is perfectly alright because

we want both. Here, to this end, we ignored any chunks that are highly dissimilar to the one represented by (1), and to reduce noise for the final LLM feed. In the case of each provider type only the first few chunks of it where taken.

VII. Results

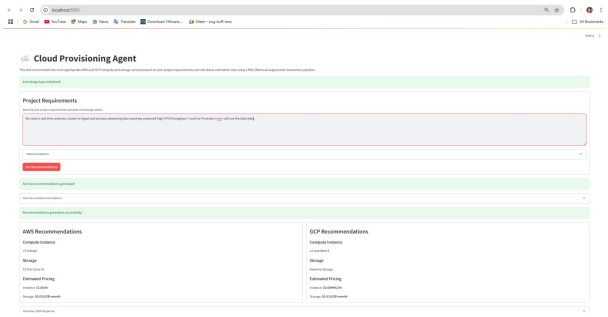

Fig. 6. Comparison of Estimated vs. Actual Cloud Costs across benchmark scenarios.

The results demonstrate that, when provided with a free-form project description, our system accurately interprets the underlying requirements and recommends the appropriate suite of services on both AWS and GCP. In our evaluation across streaming analytics, web services, batch processing, and GPU inference scenarios, the pipeline successfully extracted key parameters—such as vCPU counts, memory sizes, throughput needs, and data retention policies—and mapped them to specific instance families enabling users to obtain actionable multi-cloud estimates in a single step.

Our results support both the hypothesis that is instead of using just pretrained model finetuned model gives better results.

VIII. Future Work

Building on our prototype, we identify four key directions for enhancement:

- **Expanded Coverage:** Integrate additional clouds (e.g., Azure) and extend to more service categories (databases, CDNs, containers).
- **Live Pricing Updates:** Automate periodic ingestion from provider APIs or published feeds to keep cost data current.
- **Advanced Optimization:** Incorporate multi-cloud orchestration logic to suggest hybrid deployments that minimize combined compute, storage, and transfer costs.
- **User Customization & Feedback:** Allow inputs for enterprise discounts or compliance constraints, and continuously refine the model using real-world billing feedback.

These enhancements will improve accuracy, broaden applicability, and support dynamic, user-driven cloud planning.

IX. Conclusion

We show that a LLM, pretrained and further fine-tuned with the retrieval-augmented generation and the hybrid search, can effectively and efficiently automate the manual cloud cost estimation. This system reads natural-language descriptions of projects, and is able to retrieve up-to-the-minute pricing information, to return side-by-side cost-comparison reports that cover AWS, Azure, and GCP — removing any potential of human error or lag times. This mechanism proves the feasibility of domain-specific LLM tuning and RAG, and hybrid semantic and rule-based search, which aids in the accurate service mapping over dynamic pricing catalogs. In addition to cloud cost analysis, this pattern provides a scalable approach to transforming unstructured requirements to quantitative insights across a wide range of domains, thus generalizing the approach to problems such as resource planning, budgeting, and technical decisionsup port. This also demonstrates in action a few important technologies: finetuning LLMs on a domain task, using RAG to ground the model with up-to-date factual data, and most of all, performing hybrid (semantic + rule-based) search in an efficient way to query the knowledge store. And it's this blending of methods that allowed them to break apart and solve the tough challenge. We believe that the same pattern can be applied to other app lication areas where the textual requirements need to be quickly transformed into numerical or decision values.

References

[1] Deka Julian Arrizki, Stefanus Albert Kosim, and Umi Laili Yuhana. A comparative performance analysis and cost efficiency between aws and gcp services in cloud-based software development. In *2024 2nd International Conference on Software Engineering and Information Technology (ICoSEIT)*, pages 149–154. IEEE, 2024.

[2] Praveen Borra. Comparison and analysis of leading cloud service providers (aws, azure and gcp). *International Journal of Advanced Research in Engineering and Technology (IJARET) Volume*, 15:266–278, 2024.

[3] EROM GOEL. Evaluating scalable solutions: A comparative study of aws, azure, and gcp. 2021.

[4] M Scott Kingsley. Comparing aws, azure, and gcp. In *Cloud Technologies and Services: Theoretical Concepts and Practical Applications*, pages 381–393. Springer, 2023.

[5] Ganesh Kumar Murugesan. Cloud cost factors and aws cost optimization techniques. In *2024 12th International Symposium on Digital Forensics and Security (ISDFS)*, pages 1–7. IEEE, 2024.

[6] I Sh Nevludov and SV Sotnik. *Cloud giants: AWS, Azure and GCP*. PhD thesis, , 2023.

[7] Roman Reznikov. The economic impact of cloud technologies on the industry 4.0 development. *Economic Herald of the Donbas*, (4):74, 2023.

[8] Manish Saraswat and RC Tripathi. Cloud computing: Comparison and analysis of cloud service providers-aws, microsoft and google. In *2020 9th international conference system modeling and advancement in research trends (SMART)*, pages 281–285. IEEE, 2020.

[9] Adedamola Abiodun Solanke. Ai-enhanced finops: Predictive cost optimization across aws, azure, and gcp.

[10] Jay Tharwani and Arnab A Purkayastha. Cost-performance evaluation of general compute instances: Aws, azure, gcp, and oci. *arXiv preprint arXiv:2412.03037*, 2024.

[11] Hao Ye, Le Liang, and Geoffrey Ye Li. Decentralized federated learning with unreliable communications. *IEEE journal of selected topics in signal processing*, 16(3):487–500, 2022.